

復華風格「專業書分享」簡報自動化建置規劃

這份文件是給 **Claude Code** 逐步執行的施工規格書。使用方式：把整份 md 丟進 Claude Code，跟它說「依照這份規劃，從 Stage 0 開始執行，每完成一個 Stage 停下來讓我確認」。同目錄的 `fh_template_spec.json` 是已經從你上傳的母片抽出來的版面參數，不要讓 AI 重新猜，直接讀它。

0. 為什麼要做成 pipeline（先講設計哲學）

你現在的痛點是「直接下指令，品質常常不到標準」。根因有三個，這份規劃分別對應解掉：

痛點	根因	解法
內容空泛、抓不到書的重點	一次丟整本書，模型只能做淺層摘要	章節拆檔 + 每章獨立深讀，一次只處理一章，輸出結構化 JSON
版型跑掉、不像復華的東西	每次重新生成樣式，靠模型記憶	版型參數固定成 spec 檔，程式只填 placeholder，不自創樣式
內容深度不夠、沒有出處	只有書內資訊，沒有外部佐證	每章強制產生研究問題 → 網路查證 → evidence.json （含 URL 與檢索日期），沒有來源的論點不准上投影片

核心原則：內容產出與版面產出完全分離。所有「想內容」的階段只輸出 JSON；所有「排版」的階段只讀 JSON、不生內容。這樣任何一頁不滿意，你只要改 JSON 重跑 build，30 秒重出整份 PPT。

1. 專案規格（已確認）

- 來源書檔：PDF（可選取文字）、PDF（掃描影像，需 OCR）、EPUB / MOBI —— 三種都要支援
- 簡報規模：60 分鐘，約 45–60 頁
- 逐字稿：同時寫入 PPT 備忘稿 和 另外輸出 Word 逐字稿
- 外部研究：網路研究 + 台灣市場對照（含復華視角的延伸含意）
- 母片：FH 投影片母片_2018.pptx（4:3，10in × 7.5in）

2. 目錄結構

```
fh-book-deck/  
├── assets/  
│   └── FH_template.pptx      # 你上傳的母片，原封不動放這裡  
└── config/
```

```

|   |— fh_template_spec.json  # 版面參數（已產好，直接用）
|   |— project.yaml          # 本次專案設定（書名、講者、日期、頁數預算）
|   |— input/
|   |   |— book.pdf | book.epub  # 原始書檔
|   |— work/
|   |   |— 01_raw/                # 全書純文字 + 每頁文字
|   |   |— 02_chapters/           # ch01.md, ch02.md ... 拆好的章節
|   |   |— 03_digest/            # ch01.json ... 每章結構化摘要
|   |   |— 04_evidence/          # ch01.json ... 外部研究佐證
|   |   |— 05_deck.json          # 投影片藍圖（最重要的中間檔）
|   |   |— 06_script.json        # 逐字稿
|   |— scripts/
|   |   |— 01_extract.py
|   |   |— 02_split_chapters.py
|   |   |— 03_digest.py          # 由 Claude Code 驅動
|   |   |— 04_research.py        # 由 Claude Code 驅動
|   |   |— 05_outline.py
|   |   |— 06_build_pptx.py
|   |   |— 07_build_script.py
|   |   |— 08_qa.py
|   |— prompts/
|   |   |— digest.md
|   |   |— research.md
|   |   |— outline.md
|   |   |— narration.md
|   |— output/
|   |   |— FH_書名_分享_60min.pptx
|   |   |— FH_書名_逐字稿.docx
|   |   |— qa_report.md

```

3. 環境與相依套件（Stage 0）

bash

```

python3 -m venv .venv && source .venv/bin/activate
pip install python-pptx==1.0.2 pymupdf ebooklib beautifulsoup4 lxml \
    python-docx pyyaml pillow matplotlib pandas rapidfuzz

```

OCR（只有掃描版 PDF 需要）

macOS: brew install tesseract tesseract-lang

Ubuntu: sudo apt install tesseract-ocr tesseract-ocr-chi-tra tesseract-ocr-chi-sim

```
pip install pytesseract
```

QA 階段渲染投影片截圖用（強烈建議裝）

macOS: brew install --cask libreoffice / Ubuntu: sudo apt install libreoffice-impress

字型檢查：微軟正黑體 必須在執行機器上存在，否則 LibreOffice 渲染出來的 QA 截圖會失真（PPT 本身不受影響，但你看到的預覽會不準）。

config/project.yaml 範例：

```
yaml
book:
  title_zh: "（中文書名）"
  title_en: "（原文書名）"
  author: "（作者）"
  publisher_year: "（出版社 / 年）"
presenter:
  name: "Josh Chan"
  dept: "復華投信"
  date: "2026/XX/XX"
deck:
  minutes: 60
  target_slides: [45, 60]
  pages_per_chapter: [3, 5]    # 每章 3-5 頁內頁（不含章節頁籤）
  language: "zh-TW"
  terminology: "中文為主，專有名詞保留英文原文並於首次出現加註"
research:
  taiwan_lens: true           # 每章至少一張台灣 / 復華對照頁
  min_sources_per_chapter: 2
  max_source_age_years: 3     # 超過 3 年的數據要標註「截至 XXXX 年」
```

4. Stage 1-2：書檔解析與章節拆分

4.1 01_extract.py — 三種格式統一成純文字

輸出：work/01_raw/pages.jsonl（每行 {"page": 12, "text": "..."}）
與 work/01_raw/toc.json

```
python
# 分派邏輯
def extract(path):
    ext = Path(path).suffix.lower()
    if ext == ".epub": return extract_epub(path)    # ebooklib + BeautifulSoup
    if ext == ".pdf":
        doc = fitz.open(path)
        sample = "".join(doc[i].get_text() for i in range(min(8, len(doc))))
        if len(sample.strip()) < 200:               # 幾乎抽不到字 → 掃描版
            return extract_pdf_ocr(path)            # 300dpi 轉圖 → pytesseract lang="chi_tra+eng"
        return extract_pdf_text(path)
    raise ValueError(ext)
```

OCR 分支要注意：

- 以 `page.get_pixmap(dpi=300)` 轉圖，先做灰階 + 二值化再送 tesseract，中文辨識率差很多
- OCR 後跑一次清理：移除行尾斷字、合併被硬斷的段落、刪除頁首頁尾重複字串（用 rapidfuzz 找出現在 >60% 頁面的短字串就是頁眉頁腳）
- OCR 版務必輸出 `work/01_raw/ocr_confidence.md`，列出信心值最低的 20 頁，你要人工掃一眼，這是整條 pipeline 最容易埋錯的地方

4.2 02_split_chapters.py — 拆章

優先序（由可靠到不可靠，逐層 fallback）：

1. EPUB：直接用 `nav.xhtml / toc.ncx` 的章節節點，一個節點一個檔
2. PDF 有書籤：`doc.get_toc()` 取 level 1-2，用頁碼區間切
3. PDF 無書籤：正規式掃描章節標題行

python

```
CH_PATTERNS = [  
    r'^\s*第\s*[0-9一二三四五六七八九十]+\s*[章節課篇]',  
    r'^\s*Chapter\s+\d+', r'^\s*CHAPTER\s+\d+',  
    r'^\s*\d{1,2}\s*[\、]\s*S{2,30}$',  
    r'^\s*(前言|序言|導論|結語|後記|附錄|參考文獻)\s*$',  
]
```

加上啟發式過濾：候選行必須位於頁面上半部、該頁前後有頁面斷點、字數 < 40

輸出：`work/02_chapters/ch01_章節名.md`，每檔開頭寫 YAML front-matter：

markdown

```
---  
ch_id: ch01  
title: "第一章 ○○○"  
pages: [12, 38]  
word_count: 8421  
---  
(章節全文)
```

驗收點（Stage 2 停下來給你看）：印出章節清單表格——章號、標題、起訖頁、字數。字數異常（< 500 或 > 全書 25%）標紅，代表拆錯了。你確認後才往下走。

5. Stage 3：每章深讀 → 結構化摘要

這一步由 Claude Code 逐章執行（不是寫死的程式），但輸出格式被 schema 鎖死。

`prompts/digest.md`：

markdown

你正在為復華投信內部 60 分鐘讀書分享準備素材。

請只讀 {chapter_file} 這一章，不要引用你對這本書的既有印象，所有內容必須出自這一章的文字。

輸出 JSON，schema 如下（嚴格遵守，不要加欄位、不要 markdown 圍欄）：

```
{
  "ch_id": "ch01",
  "title": "章節標題",
  "one_line": "一句話講完這章在說什麼（30 字內）",
  "thesis": "這章的核心主張（80 字內，要是一個可被反駁的論點，不是描述）",
  "key_points": [
    {"point": "論點標題（12 字內）",
      "elaboration": "展開說明（100–150 字）",
      "book_evidence": "書中支持這論點的具體證據 / 案例 / 數據，要能對應原文",
      "page_ref": "原書頁碼或頁碼區間"}
    // 3–5 個
  ],
  "quotes": [{"text": "值得上投影片的原文引句（50 字內）", "page_ref": "p.34"}], // 1–2 個
  "data_points": [{"claim": "書中提到的數據", "value": "數字", "as_of": "書中標示的年份", "page_ref": ""}],
  "counterpoint": "這章最站不住腳、或最值得挑戰的地方（80 字內）",
  "open_questions": ["需要外部資料才能回答的問題"] // 2–3 個，這會餵給下一階段
}
```

品質底線（任何一項不符就重寫）：

- key_points 的 elaboration 不能是 thesis 的改寫，必須提供 thesis 沒有的資訊
- book_evidence 不可留空、不可寫「書中提到」這種空話，要有具體內容
- counterpoint 不准寫「本章論述完整」之類的敷衍句
- open_questions 必須是「查得到資料的具體問題」，不是哲學提問

執行方式：對每個 work/02_chapters/*.md 跑一次，寫入 work/03_digest/chNN.json。Claude Code 應該一次處理一章、寫檔、再處理下一章，不要一次把所有章節塞進 context。

6. Stage 4：外部研究擴編（台灣市場對照）

prompts/research.md 的骨幹：

markdown

輸入：{digest_json}（某一章的摘要）

任務：把這一章的論點連結到「可查證的外部現實」與「台灣市場 / 復華投信的視角」。

步驟：

1. 從 open_questions + data_points 生成 3–5 個檢索查詢（中英各半）
2. 用 WebSearch 檢索，優先級：官方統計 > 監理機關 > 學術 / 研究機構 > 財經媒體 > 部落格
- 台灣類優先來源：主計總處、金管會、中央銀行、投信投顧公會、證交所、TEJ、國發會
3. 只採信能給出明確 URL 與日期的資料

輸出 JSON：

```
{
  "ch_id": "ch01",
  "verified": [{"book_claim": "書中說法", "status": "confirmed|outdated|contested",
    "current_fact": "最新事實與數字", "as_of": "2026-XX", "source_title": "", "source_url":
    ""}],
  "taiwan_lens": [{"angle": "台灣 / 復華視角的對照標題",
    "insight": "150 字內，說明這章的觀點在台灣市場會怎麼呈現 / 有什麼不同",
    "supporting_data": "具體數據", "source_title": "", "source_url": ""}],
  "extensions": [{"title": "延伸論點", "content": "書沒講但值得補的", "source_title": "",
    "source_url": ""}],
  "chart_candidates": [{"title": "可畫成圖表的資料", "data": [{"label": "", "value": 0}],
    "chart_type": "bar|line|stacked", "source_title": "", "source_url": ""}]
}
```

硬性規則：

- 每一筆都必須有 source_url。查不到就不要寫，寧可少一筆，絕對不准編造 URL 或推測數字。
- 標示 outdated 的書中數據，一定要放進投影片（這是分享會最有價值的部分）。
- 每章至少 1 筆 taiwan_lens。

輸出 work/04_evidence/chNN.json。驗收點：跑 08_qa.py --check-sources，把所有 URL 做一次 HTTP 檢查，死連結列出來。

7. Stage 5：投影片藍圖 deck.json

這是整條 pipeline 的核心中間檔。05_outline.py 把 digest + evidence 合成藍圖，再由 Claude Code 依 prompts/outline.md 潤飾文案。

7.1 頁數預算（60 分鐘 / 45–60 頁）

區塊	版面	頁數
封面	標題 (0)	1
全書地圖 / Agenda	目錄 (1)	1–2
為什麼讀這本書	內頁 1-1 (4)	1

每章：章節頁籤	頁籤 (2)	$1 \times N$
每章：核心主張	內頁1-1 (4)	$1 \times N$
每章：論點展開	內頁1-1 (4)	$1-2 \times N$
每章：台灣對照 / 數據	內頁2-1 (6) 或 空白內頁 (9)	$1 \times N$
全書綜合	內頁3-1 (8)	2-3
可落地的行動	內頁3-1 (8)	1-2
我的異議	內頁2-1 (6)	1
結語 / Q&A	結尾 (10)	1

以 8 章書估： $1+2+1+8 \times 4+3+2+1+1 = 43-59$ 頁，符合目標。若書超過 10 章，在 05_outline.py 自動合併相鄰章節成「主題群組」，每群組共用一個頁籤頁。

7.2 deck.json schema

```

json
{
  "meta": {"book_title": "", "presenter": "Josh Chan", "dept": "復華投信", "date": "2026/XX/XX"},
  "slides": [
    {
      "id": "s001",
      "layout": 4, // fh_template_spec.json 的 layout index
      "kind": "cover|toc|divider|content|chart|closing",
      "title": "主標（18 字內）",
      "subtitle": "副標（28 字內，可為 null）",
      "body": [
        {"level": 0, "text": "第一層要點（40 字內）"},
        {"level": 1, "text": "第二層補充（60 字內）"}
      ],
      "chart": null, // 或 {"type": "bar", "data": [...], "title": ""}
      "sources": [{"label": "書名 p.34"}, {"label": "主計總處", "url": "https://..."}],
      "narration": "這一頁的逐字稿（見 Stage 7）",
      "duration_sec": 70,
      "ch_id": "ch01"
    }
  ]
}

```

7.3 文案硬規則（寫進 **prompts/outline.md**，這是解決「品質不到標準」的關鍵）

【標題】

- 主標一律寫成「有結論的句子」，不是名詞標籤
 - ✗ 「風險管理」 ✓ 「風險預算該分配在部位，不在情緒」
- 主標 ≤ 18 字；副標 ≤ 28 字，副標負責補「為什麼 / 所以呢」

【內文】

- 每頁第一層要點 3–5 條，每條 ≤ 40 字，不寫完整句號結尾的長句
- 層級最多 2 層，不准出現第 3 層
- 每頁總字數 ≤ 160 字（4:3 版面，24pt 內文，超過就爆版）
- 每頁至少有一個「具體物」：數字、年份、案例名、金額、比例。全部都是抽象概念的頁面一律重寫

【資料來源】

- 每一頁都要有 sources，書內容寫「《書名》 p.XX」，外部資料寫「機構名（YYYY/MM）」
- 外部資料同時保留 url 進 deck.json，供 QA 檢查，但投影片上只印機構名與日期

【語言】

- 繁體中文為主，專有名詞保留英文原文，首次出現寫「中文（English）」
- 不使用「賦能、抓手、閉環、顆粒度」這類對岸用語

8. Stage 6 : 06_build_pptx.py — 產出投影片

8.1 必備工具函式

```
python
from pptx import Presentation
from pptx.util import Emu, Pt
from pptx.dml.color import RGBColor
import copy, json

SPEC = json.load(open("config/fh_template_spec.json", encoding="utf-8"))

def delete_all_slides(prs):
    """母片檔內建 11 張示範頁，必須先清空。python-pptx 沒有 API，要動 XML。"""
    xml_slides = prs.slides._sldIdLst
    for sld in list(xml_slides):
        rId = sld.get('{http://schemas.openxmlformats.org/officeDocument/2006/relationships}id')
        prs.part.drop_rel(rId)
        xml_slides.remove(sld)

def add(prs, layout_idx):
    return prs.slides.add_slide(prs.slide_layouts[layout_idx])

def set_ph(slide, idx, text, size_pt=None, color=None):
```

```

"""依 placeholder idx 填字，並強制設定中文字型。"""
ph = slide.placeholders[idx]
tf = ph.text_frame
tf.clear()
lines = text if isinstance(text, list) else [{"level": 0, "text": text}]
for i, ln in enumerate(lines):
    p = tf.paragraphs[0] if i == 0 else tf.add_paragraph()
    p.level = ln.get("level", 0)
    r = p.add_run(); r.text = ln["text"]
    if size_pt: r.font.size = Pt(size_pt)
    if color: r.font.color.rgb = RGBColor.from_string(color)
    set_ea_font(r, "微軟正黑體") # 關鍵！見下
return ph

```

```

def set_ea_font(run, name):
    """python-pptx 只會設 latin 字型，中文要直接插 <a:ea>。"""
    rPr = run._r.get_or_add_rPr()
    for tag in ("a:ea", "a:cs"):
        el = rPr.find(qn(tag))
        if el is None:
            el = rPr.makeelement(qn(tag), {}); rPr.append(el)
        el.set("typeface", name)

```

```

def add_source_line(slide, sources):
    c = SPEC["custom_elements"]["source_line"]
    box = slide.shapes.add_textbox(*[Emu(v) for v in c["emu"]])
    labels = " ; ".join(s["label"] for s in sources)
    r = box.text_frame.paragraphs[0].add_run()
    r.text = f"資料來源：{labels}"
    r.font.size = Pt(c["size_pt"]); r.font.color.rgb = RGBColor.from_string(c["color"])
    set_ea_font(r, "微軟正黑體")

```

8.2 版面對應表（直接用 fh_template_spec.json）

layout	名稱	placeholder idx
0	標題（封面）	0=主標 48pt B82837 / 14=副標 20pt / 13=部門日期
1	目錄	13=條列
2	頁籤（章節分隔）	14=章節名 44pt B82837
3	內頁1	13=主標 40pt / 14=內文 24pt
4	內頁1-1（主力版型）	13=主標 36pt / 14=副標 24pt C9A063 / 15=內文 24pt
5 / 6	內頁2 / 2-1	同上結構，樣式變體
7 / 8	內頁3 / 3-1	同上結構，樣式變體
9	空白內頁（圖表用）	14=內容區；需先刪掉裝飾群組 shape

10 結尾	14=結語文字
-------	---------

8.3 圖表頁

`chart_candidates` 用 `matplotlib` 出圖再嵌入（不用 PPT 原生圖表，中文字型與配色比較好控）：

```
python
FH_COLORS = ["#B82837", "#939396", "#B08F6E", "#00A0E9", "#262627"]
plt.rcParams["font.sans-serif"] = ["Microsoft JhengHei", "PingFang TC", "Noto Sans CJK TC"]
plt.rcParams["axes.unicode_minus"] = False
```

存 PNG 200dpi，寬 8208144 EMU 等比縮放後 `add_picture` 到 layout 9

圖表規範：主數列一律用復華紅 `#B82837`，對照組用灰 `#939396`；不畫圓餅圖；座標軸單位寫清楚；圖下方仍要有資料來源行。

8.4 溢排保護

內文 placeholder 高度 4,537,075 EMU (≈ 3.72 in)。建置時估算：24pt 中文一行約 18 字（寬 8,208,144 EMU），行高約 0.45 in → 最多約 8 行。在 `06_build_pptx.py` 加入 `estimate_lines()`，超過 8 行就：先降到 20pt → 還是超過就把該頁拆成兩頁並在主標後加「(續)」。絕不允許自動縮到 16pt 以下。

9. Stage 7：逐字稿（備忘稿 + Word）

`prompts/narration.md`：

markdown

為 `deck.json` 中的每一頁寫口語逐字稿。

規則：

- 語速基準：中文每分鐘 220 字。依 `duration_sec` 反推字數（70 秒 $\approx$ 250 字）
- 全簡報總時長控制在 55 分鐘（留 5 分鐘 Q&A）
- 口語，不是書面語。可以用「這邊」「其實」「我自己的經驗是」
- 每頁結構：① 承接上一頁的一句過場 ② 這頁的核心 ③ 展開 / 舉例 ④ 拋給下一頁的鉤子
- 投影片上已經寫的字不要逐條唸出來，逐字稿要講的是「投影片上沒寫的那一層」
- 數據頁必須把數字唸清楚並說明單位與時間點
- 章節頁籤頁的逐字稿只要 20–30 秒的過場
- 標註 [停頓] [看向聽眾] [這裡可能會被問：...] 等提示

輸出兩份：

1. 寫回 `slide.notes_slide.notes_text_frame` (`06_build_pptx.py` 一併處理)

2. 07_build_script.py 用 python-docx 產 output/FH_書名_逐字稿.docx :
 - 表格式版面：左欄頁碼 + 投影片主標，右欄逐字稿
 - 每章起始插入分頁
 - 頁首寫累計時間軸（「第 12 頁 / 累計 14:30」）
 - 文末附「預期提問與備答」一節（從各章 counterpoint 生成）

10. Stage 8 : 08_qa.py — 自動品管（不可略過）

輸出 output/qa_report.md，以下任何一項 FAIL 就不准交付：

檢查	判準
頁數	$45 \leq \text{total} \leq 60$
溢排	每頁內文估算行數 ≤ 8 ；主標 ≤ 18 字；副標 ≤ 28 字
資料來源	每一頁（封面 / 頁籤 / 結尾除外）都有 sources，且非空字串
外部連結	所有 url HTTP 200；死連結列出
具體性	每頁 body 至少含一個數字 / 年份 / 專有名詞；否則標 WARN
版型純度	每張 slide 的 layout name 必須在模板 11 種之內；無自建 textbox（資料來源行除外）
字型	掃描所有 run，a:ea typeface 必須是 微軟正黑體
逐字稿	每頁 narration 字數 = $\text{duration_sec} \times 220/60 \pm 25\%$ ；總時長 50–58 分鐘
對岸用語	黑名單掃描（賦能 / 抓手 / 閉環 / 顆粒度 / 打法 / 復盤 / 落地性...）
視覺	LibreOffice 轉 PNG 全頁截圖，輸出到 output/preview/，供你翻一次

```
bash
soffice --headless --convert-to pdf output/*.pptx --outdir output/
pdftoppm -png -r 80 output/*.pdf output/preview/slide
```

11. 執行順序（給 Claude Code 的指令腳本）

```
bash
# Stage 0
make setup          # 建目錄、裝套件、複製母片到 assets/

# Stage 1–2 解析與拆章
python scripts/01_extract.py --input input/book.pdf
python scripts/02_split_chapters.py
# → 停：人工確認章節清單表格
```

Stage 3 逐章深讀 (Claude Code 逐檔執行 prompts/digest.md)

→ 停：抽 2 章看 digest 品質

Stage 4 外部研究 (Claude Code 逐章執行 prompts/research.md + WebSearch)

`python scripts/08_qa.py --check-sources`

→ 停：確認來源可信度

Stage 5 藍圖

`python scripts/05_outline.py` # 先算頁數預算與版面分派

→ Claude Code 依 prompts/outline.md 潤飾 work/05_deck.json 文案

→ 停：你直接讀 deck.json 改字，這是最省時的修改點

Stage 6–7 建置

`python scripts/06_build_pptx.py`

`python scripts/07_build_script.py`

Stage 8 品管

`python scripts/08_qa.py --all`

改稿迴圈：任何不滿意 → 改 work/05_deck.json → 重跑 Stage 6–8。不要重跑前面的階段。

12. 三個最容易翻車的地方 (請 Claude Code 特別注意)

1. 母片內建 11 張示範頁沒清乾淨 → 成品開頭多出 11 張空白頁。
`delete_all_slides()` 必須在 `add_slide` 之前呼叫，且要 `drop_rel`。
2. 中文字型只設了 **latin** → 在別台電腦開會變新細明體。所有 run 都要過 `set_ea_font()`，QA 有對應檢查。
3. 研究階段編造來源 → 這是唯一會讓你在會議上出事的錯誤。research prompt 的「查不到就不要寫」是硬性規則，QA 的 URL 檢查是第二道關卡，兩道都要留著。

13. 下一步

書的電子檔準備好之後，把 `input/` 放進去就能開跑。如果你要，我可以先把 `scripts/` 底下 8 支程式的完整程式碼一次寫出來給你，你 clone 下來就能執行 —— 跟我說一聲即可。